# Contents

**Week 32 — The direct model, learned from zero** **1**
0. The story in eight sentences . . . 1
1. Where we were, and what was asked . . . 2
2. The two models side by side . . . 2
3. The direct model, piece by piece . . . 3
4. The design, and the reason for every decision . . . 6
5. What was built — the code map . . . 7
6. The checks before training — and the three things they taught us . . . 8
7. The 24 training runs — facts . . . 9
8. The scoring gates — and the float32 lesson (G3) . . . 9
9. Results — how to read the table . . . 10
10. What it means . . . 11
11. Limits and open items (say them before they are asked) . . . 12
12. Questions you will be asked — and the answers . . . 12
13. Numbers to memorise . . . 13
14. The logical flow for tomorrow's deck (one idea per slide) . . . 14

# Week 32 — The direct model, learned from zero

*For Tanmay, before the meeting of Thu 29 Oct 2026. Every number below comes from the Torch runs of 27–28 Oct (fingerprint 21152059, training 21158520, audit 21225432) and the repository at commit `898f966` on `feat/direct-model`. Nothing is rounded beyond what the tables show.*

---

## 0. The story in eight sentences

This is the spine. Every slide tomorrow is one of these sentences with one picture or one table under it. If a question pulls you off the spine, answer it and come back to the sentence you were on.

1. Until now our fast model needed a dice roll: read the flood map, draw which sites the water knocks out, run the network once per draw, average 100 draws.
2. Dr. Lin asked whether one pass could do it: flood map in, every site's failure probability out, no drawing.
3. We could test that without new simulation, because the 1,000 runs we already have for each of 70 storms give exactly the target such a model needs — the share of runs in which each site has failed by 6, 24, 48 and 96 hours.
4. We built it: the same network, fed the depth, the fragility probability and a storm-size summary instead of a seed draw, with its output composed so that a site's probability can never fall below its direct-flooding probability.
5. We wrote down seven predictions before training, trained 24 models (four label/input cells, three training sizes, two random starts), and scored them on the same 13 held-out storms as last week.
6. Result one: the one-pass model matches the 100-draw model on all four cells (13-storm mean loss 0.069 vs 0.071 under static labels), at 11 ms per storm instead of about a second, and its storm sizes are within 3 % of the simulator's on every held-out storm.
7. Result two: adding GISSR storms stopped helping at 34 — the 17→34 gain is entirely from covering water levels the old 17 storms lacked; 34→50 added nothing. The pre-registered reading of that pattern is "map diversity, not map count, is the limit".
8. Where it falls short: under arrival-time labels the loss sits 25–40 % above the simulator's own noise floor, and the one storm with a different spatial shape (GeoClaw) gets worse under arrival labels as more GISSR storms are added — which is the spatial-generalisation problem showing itself in the only storm that can show it.

Three of seven predictions passed. Two of the misses are because the model did better than predicted

(vs the lookup under arrival labels, vs the two-stage model everywhere); one is the named failure mode (the flat curve); one is a replicability check that fails exactly where two things are tied.

---

## 1. Where we were, and what was asked

**The two-stage surrogate (Weeks 19–31).** One flood map → Hazus fragility gives each site a probability φ_i of being knocked out directly → draw a seed set (each site independently with probability φ_i) → CascadeGNN, given the seed bits, predicts every site's probability of having failed by 6/24/48/96 h → repeat for 100 seed sets and average. The averaging is what turns "given these seeds" into "given this flood". Last week's numbers (storm-summary variant, 57 training storms, 13 held-out): per-run non-seed BCE 0.054 / 0.054 / 0.109 / 0.099 in the four cells; ahead of the nearest-storm lookup on 12 of 13 storms in every cell.

**Dr. Lin, 15 Oct.** First the leakage question about the storm summary (answered: it is computed from the seed bits, no cascade outcome). Then the proposal: a *direct* model — flood map in, per-site failure probability out, one forward pass, no Monte Carlo. His conditions: the generalisation target is *across flood maps*, not across seed draws; it needs "a few hundred flood conditions"; it will be harder to train.

**Our reply (15 Oct), which became this week's experiment.** Agree, and we can start without new simulation. The simulator has already run 1,000 times on each of 70 storms; the share of runs in which a site has failed by t_k is a Monte Carlo estimate of exactly the probability the direct model should output. So: train on the storm-level probabilities of 50 storms, validate on 7, test on the same 13 as last week, compare against the simulator, the two-stage model and the lookup, and draw the training-size curve 17 / 34 / 50 to see whether "more maps" is the limit. The one thing we flagged in advance: the GISSR maps are nested by water level (rank correlation 0.9998 between level and wet-site count over 593 sampled storms, from Week 29), so they add levels and timing, not spatial variety; the real prerequisite might be diversity of maps rather than their number.

**22 Oct meeting.** Split by storm agreed; predictions as drafted; flood maps: the lab has GISSR only and can generate more from it.

---

## 2. The two models side by side

| | Two-stage (Weeks 19–31) | Direct (this week) |
|---|---|---|
| One training example | one simulator **run** (one seed draw, one cascade) | one **storm** |
| Input per site | 8 attributes, storm summary from the *realised* seed count, **seed bit**, timing | 8 attributes, **depth**, **φ_i**, storm summary from the *expected* seed count Σφ, timing |
| Target per site and horizon | 0/1: did this site fail by t_k in this run | $\hat{\pi}$_ik: share of 1,000 runs in which it failed by t_k |
| What the network learns | the cascade given a particular seed set | the cascade averaged over seed sets, as a function of the map |
| Output | P(failed by t_k \| these seeds) | P(failed by t_k \| this flood) = 1 − (1 − φ_ik)(1 − g_ik) |
| Loss | BCE against 0/1 labels | soft BCE against $\hat{\pi}$ (same formula, probability in place of the label) |
| To predict for a new map | sample 100 seed sets, 100 passes, average (≈ 1 s) | one pass (≈ 11 ms) |
| Training examples | 45,600 runs (57 storms) | 50 storms |

| | Two-stage (Weeks 19–31) | Direct (this week) |
|---|---|---|
| Steps per epoch / epochs | 45,600 / 6 | 50 / up to 600 with early stopping |

The network itself — projection, two layers of relation-wise attention, per-type head, hazard read-out — is byte-identical. What changed is what goes in, what comes out, and what it is scored against.

---

## 3. The direct model, piece by piece

### 3.1 One example is one storm

In `src/gnn/train.py::run_train_marginal` the training loop is:

```
for epoch in range(max_epochs):
    shuffle(train_storms)                    # 17, 34 or 50 of them
    for storm in train_storms:
        g = model(x[storm], edges)           # one pass on the whole 6,231-site graph
        logit_p = compose_with_fragility(g, phi_k[storm])
        loss = soft_bce(logit_p, pi_hat[storm])   # mean over 6,231 x 4 entries
        loss.backward(); adam.step()
    val_loss = mean over the 7 validation storms
    keep best.pt; stop when val has not improved for 50 epochs
```

One gradient step per storm, 50 steps per epoch, at most 30,000 steps. The two-stage model took 273,600 steps (45,600 runs × 6 epochs). A direct-model run took 5–25 minutes on an L40S.

### 3.2 The inputs — worked example on `syn_ts_914_6_1p2955` (2.17 m, the small old held-out storm)

Every site i gets a row of width **17** (or **23** in timing cells):

```
[ x_i (8) | min(d_i, 5 m)/5 | φ_i | c (7, or 10 with timing) | q_i (3, timing cells) ]
```

- **x_i (8)**: the frozen graph's attributes (lat, lon, two constant placeholders, four type-specific columns, scaled to [0, 1]). Unchanged from every campaign.
- **depth**: the site's water depth from `data/flood/<table>_node_depths_v1.csv`, capped at 5 m and divided by 5. Capped because every fragility curve is saturated long before 5 m, so deeper water carries no information and would only stretch the input range. For `subway_coney_island_stillwell_av_dfnq` in 914_6: depth 0.203 m → input 0.0407. *The two-stage model never saw depth at all* (its depth column is a constant placeholder); here the flood enters the network directly.
- **φ_i**: the Hazus seed probability at that depth, computed by `src/simulation/fragility.py::failure_probabilit` the same function the simulator draws seeds with. Section 3.3.
- **c**: the expected-seed summary, the same 7 (or 10) numbers for every site in the storm. Section 3.4.
- **q_i** (timing cells only): arrival hour, flooded duration, time to peak, each divided by 96. For the Coney Island station in 914_6: arrival 7.8 h, duration 2.6 h, time-to-peak 0.9 h → 0.081, 0.027, 0.010.

No seed bit. Nothing in the input is drawn at random. The input is a deterministic function of the map (and the timing table), which is what makes one pass enough.

Storm facts for 914_6 you should be able to recite: 156 wet sites (121 telecom clusters, 15 fuel, 9 water, 6 subway, 5 power, no hospital); Σφ = 91.3 expected seeds (the frozen record's run 19 had 92; the seed frequency over 1,000 runs sums to 91.6); the simulator's mean failed counts are 190.6 / 258.5 / 290.0 / 313.3 at 6 / 24 / 48 / 96 h — so by hour 6 the cascade has already doubled the damage, and by hour 96 total failures are 3.4× the seeds.

### 3.3 Fragility φ — the thing the simulator draws from

> **Box.** φ_i = Φ( (ln d_i − ln δ_a) / β_a ), with (δ_a, β_a) per system: power (0.6, 0.5), telecom (0.3, 0.6), hospital (0.6, 0.5), subway (0.1, 0.4), water (0.5, 0.5), fuel (0.5, 0.6). φ_i = 0 for dry sites and for the 99 external sites (which never seed). Regime `pluvial`, scaling 1.0, as in every label campaign.
>
> **By hand.** Telecom cluster at 0.222 m: z = (ln 0.222 − ln 0.3)/0.6 = (−1.5051 + 1.2040)/0.6 = −0.502 → φ = Φ(−0.502) = 0.308. Subway station at 1.0 m: z = (0 + 2.3026)/0.4 = 5.76 → φ = 0.999999996, which float32 stores as exactly 1.0 (this matters in Section 8).

Why φ is an input and not just depth: the network could in principle learn the curve from depth, but φ is the quantity the simulator actually uses, per system, and giving it directly means the composition in 3.7 is exact rather than learned. The fingerprint's F3 check (Section 6.3) is precisely the test that our φ is the simulator's φ.

### 3.4 The expected-seed summary — why it exists and how it differs from last week's

Week 29's lesson: the network passes messages over two links, so a site with no flooded site within two links receives the same input in every storm and cannot tell a small storm from a large one. The fix was a storm-level summary appended to every site. Last week it was built from the *realised* seed draw: c_a = log(1 + k_a)/log(1 + N_a), k_a the number of seeds in system a. The direct model has no draw, so the count is replaced by its expectation:

> **Box.** c_a = ln(1 + Σ_{i∈a} φ_i) / ln(1 + N_a) for each of the six systems, plus the same over all sites; in timing cells, plus the mean of the three timing columns over wet sites. `src/gnn/data.py::storm_context_expected`.
>
> **914_6:** power Σφ = 0.11 of 203 → 0.019; telecom 75.27 of 4,150 → 0.520; hospital 0 → 0; subway 4.86 of 493 → 0.285; water 4.81 of 137 → 0.357; fuel 6.27 of 1,187 → 0.280; overall 91.3 of 6,231 → ln(92.3)/ln(6,232) = 4.525/8.737 = 0.518. **t198w16 (4.72 m):** overall Σφ = 660.1 → 0.743. **geoclaw_2050:** 295.5 → 0.651.

The log scaling compresses: a storm with seven times the expected seeds (660 vs 91) gets a summary 0.74 vs 0.52. That compression is the suspected reason the two-stage model over-counted the smallest storms by 23–31 % (open item from Week 31); it is carried over unchanged here so the comparison with B2 is like for like.

### 3.5 Timing, and what "arrival labels" means for φ

Two label campaigns exist for every storm. *Static:* all seeds are stamped at hour 0 and the cascade runs from there. *Arrival:* each seed is stamped at the hour the water reaches it, on a clock whose zero is the Battery peak; the cascade runs from the first arrival; horizons 6/24/48/96 h are counted from the peak.

For the direct model this means the seed part of the probability can itself depend on the horizon: a site whose water arrives after t_k cannot be a seed "by t_k". So `phi_by_horizon` sets φ_ik = φ_i under static labels and φ_ik = φ_i · 1[arrival_rel ≤ t_k] under arrival labels. Honest note to carry: in every storm we have, every wet site's water arrives before the peak (time_to_peak_h ≥ 0 for all wet sites in the timing table), so arrival_rel ≤ 0 ≤ 6 h and the indicator is always 1. The rule is in the code because it is the right rule; on these maps it changes nothing. If someone asks "what does arrival do to φ?", that is the answer.

What arrival labels *do* change is the target: more sites fail, and later. For geoclaw_2026 the mean failed count by 96 h is 1,970.6 under static and 2,199.4 under arrival; for geoclaw_2080 2,509.1 vs 3,681.4.

### 3.6 The network and the hazard read-out

Unchanged (`src/gnn/model.py::CascadeGNN`): per-type linear projection to 64, two `HeteroConv` layers of `TransformerConv` with 4 heads over the 20 relations, residual + ReLU, per-type MLP head to 4 numbers,

read as hazard logits η_k:

> h_k = σ(η_k) = P(fails in window k | survived to its start); S_k = Π_{j≤k}(1 − h_j); g_k = 1 − S_k, non-decreasing in k by construction. The head returns logit(g_k).

Parameter counts: 699,032 (width 17) and 701,336 (width 23). Only the input projections changed size.

### 3.7 The composition — the one new equation

> **Box.** For site i and horizon k:
>
> p_ik = 1 − (1 − φ_ik)(1 − g_ik)
>
> Read it as: the site has failed by t_k if the water knocked it out directly (probability φ_ik) **or**, if not, the cascade reached it by t_k (probability g_ik, given it was not a seed). By the law of total probability, P(failed by t_k) = φ + (1 − φ)·P(failed by t_k | not a seed), so **g is exactly the conditional cascade probability** and the network learns only that part; the seed part is supplied, not learned.
>
> **Properties.** p ≥ φ always (the second factor is ≤ 1). p is non-decreasing in k because both φ_ik and g_ik are. Where φ = 1 (deep subway station), p = 1 regardless of g; where φ = 0 (dry site), p = g.
>
> **Worked example.** A telecom cluster with φ = 0.308 (0.222 m) whose cascade probability by 24 h is g = 0.5: p = 1 − 0.692 × 0.5 = 0.654. If instead the network said g = 0 (no cascade reaches it), p = 0.308: the floor set by the flood itself.

**Stable form in code** (`compose_with_fragility`): because log(1 − σ(z)) = −softplus(z),

log(1 − p) = log(1 − φ) − softplus(logit g), then logit(p) = log(1 − e^{log(1−p)}) − log(1 − p).

Two clamps: φ is kept ≤ 1 − $10^{-6}$ before the log, and log(1 − p) ≤ $-10^{-6}$ so the logit stays finite. Those two clamps are the whole of the G3 story in Section 8.

### 3.8 The target

> **Box.** π̂_ik = (1/1000) · #{runs r : T_ir ≤ t_k}, where T_ir is the hour at which site i failed in run r (seeds included; the same ≤ rule as `build_labels`). `src/gnn/data.py::marginal_labels` also stores the seed frequency per site and the two half-sample estimates (runs 1–500 and 501–1,000) for the floor.

π̂ is an estimate of the true marginal π with binomial noise: standard deviation √(π(1−π)/1000) ≤ 0.016, largest at π = 0.5. That noise is why a perfect predictor cannot score zero, and why the floor in 3.9 exists.

Targets were built once per label mode by the fingerprint job into `$SCRATCH/results/gnn_direct_targets_v1/{static,` (70 files each); training reads only that cache and refuses to rebuild from label files, so no training job ever touched the 108–364 MB label JSONs.

### 3.9 The loss, its minimum, and the floor

> **Box.** Soft log loss: L = −(1/4N) Σ_{i,k} [ π̂_ik ln p_ik + (1 − π̂_ik) ln(1 − p_ik) ]. In code, with z = logit p: π̂·softplus(−z) + (1 − π̂)·softplus(z), averaged.
>
> Minimum: differentiate in p: −π̂/p + (1 − π̂)/(1 − p) = 0 ⇔ p = π̂. The value there is the binary entropy H(π̂) = −π̂ ln π̂ − (1 − π̂) ln(1 − π̂). So **the best possible loss is the entropy of the target, not zero**, and it differs from storm to storm (more sites in the uncertain middle → higher entropy).
>
> **Worked example.** One site, π̂ = 0.3. At p = 0.3: 0.3·(−ln 0.3) + 0.7·(−ln 0.7) = 0.361 + 0.250 = 0.611. At p = 0.4: 0.3·(−ln 0.4) + 0.7·(−ln 0.6) = 0.275 + 0.358 = 0.633. At p = 0.03 (ten times too low): 0.3·3.507 + 0.7·0.0305 = 1.073. Over-confidence in the wrong direction is punished much harder than being off by a tenth.

**Why this loss and not squared error:** it is the same family as the two-stage loss (BCE with the 0/1 label replaced by its expectation), it is a proper scoring rule (minimised only at the truth), and it gives the entropy floor for free. Also: for a *fixed* p, the per-run BCE averaged over runs equals the

soft BCE against π̂ exactly — so the two losses are the same object once the seeds are integrated out.

**The split-half floor.** Score π̂ from runs 1–500 against π̂ from runs 501–1,000 and the reverse, average. This is the loss a predictor that knew the simulator's answer would still pay for the target's sampling noise. To second order it is $H(\pi) + \mathrm{Var}(\hat{\pi}_B)/(2\pi(1-\pi)) = H(\pi) + 1/1000$ per uncertain site — the floor is the entropy plus about 0.001. Static 13-storm mean floor 0.0642; arrival 0.0840.

**Why 0.069 here cannot be compared with last week's 0.054.** Last week's number is the per-run BCE of a model that *sees the realised seeds*; the uncertainty it has to cover is H(Y | seeds), which is smaller than H(Y). This week's is at storm level with no seed knowledge. The right bridge is B2: the same two-stage checkpoint, seeds integrated out by averaging 100 draws, scored at storm level — and it scores 0.0707, not 0.054. Same model, two questions, two numbers.

---

## 4. The design, and the reason for every decision

### 4.1 Split by storm: 50 / 7 / 13

The generalisation target is a new map, so storms must not leak across the split. Training 50 = the 17 old training storms + 33 new; validation 7; test 13 = the two-stage experiment's 13 held-out storms, untouched, so every comparison with last week is on the same storms.

**Why the 7 validation storms come from the new set only, and how they were picked:** the 40 new training storms sorted by target water level, every 5th from the 4th (positions 4, 9, …, 34) → 2.14, 2.48, 2.81, 3.15, 3.49, 3.83, 4.16 m, alternating slow and fast (the manifest alternates lead classes, so a fixed stride mixes them). Taking them from the new storms leaves the old 17 intact for the 17-storm arm; spreading them by level means early stopping judges the whole range. Validation is used for early stopping only; no verdict reads it.

### 4.2 The curve 17 / 34 / 50 — and the correction from 17/37/57

Nested sets: 17 = the old storms; 34 = 17 + every other of the 33 new by level (17 added, 1.94 → 4.57 m); 50 = all. The design note of Week 31 said 17/37/57; that counted the 7 validation storms as training data. Caught on 27 Oct before the freeze; the frozen script says 17/34/50. Same 7 validation storms and the same early-stopping rule in every arm, so the curve varies one thing.

### 4.3 Training settings

Adam 1e-3, weight decay 1e-5, dropout 0.1, hazard head — identical to the two-stage runs so the comparison is about inputs and target, not optimisation. Up to 600 epochs, early stopping on the 7-storm validation loss with patience 50, best-validation checkpoint. Split seed 42 (storm order); PyTorch seeds 1 and 2 (weights, dropout). Two seeds because last week showed a single start can mislead on tight comparisons.

### 4.4 Four cells

{static, arrival labels} × {without, with the per-site timing columns}. "plain" in the file names means *no timing columns*; the expected-seed summary is always on. There is no no-summary arm, because Week 29 settled that question. 4 cells × 3 sizes × 2 seeds = 24 runs.

### 4.5 Baselines — what each one answers

- **B0, fragility only, p = φ.** "Is the network doing anything?" Any cascade at all beats it. It scores 0.38 on the smallest storm and 3.5 on the largest: without a cascade you under-predict catastrophically.

- **B1, lookup by expected seed count.** "Could a table do this?" For a test storm, take the training storm whose Σφ is closest and answer with its π̂. Why Σφ: it is the direct-model analogue of the two-stage lookup (which matched on the run's realised seed count), and it needs no draw. For 914_6 (Σφ 91.3) it picks `syn_ts_708_13_1p1567` (89.1). Its weakness, visible in the table: it picks `geoclaw_2026` for `syn2_t934c34` (205.4 vs 206.4 expected seeds) and a GISSR storm for `geoclaw_2050` — same seed count, different shape, loss 0.41 and 0.31.
- **B2, the two-stage model as it would be used.** "Does one pass lose anything against a hundred?" The `mix57_full_ctx_<cell>_s<seed>` checkpoint, 100 seed sets drawn from φ, average, scored at storm level. Computed inside the audit (not through `predict_cascade.py`, which reads hazard-head checkpoints with the wrong read-out — an open item), with gate G6 checking that the inputs assembled there equal `data.py`'s own build bit for bit. Note B2 was trained on 57 storms — the 50 plus our 7 validation storms — so it had slightly more data.
- **Floor.** Section 3.9. "How far from perfect is this?"

### 4.6 Metrics E1–E5 (per storm, against π̂)

E1 soft log loss, mean and per horizon. E2 mean $|p - \hat{\pi}|$ over all sites and over sites with $\hat{\pi} > 0.05$ (the dry majority would hide everything otherwise). E3 count error $|\Sigma p - \Sigma\hat{\pi}|/\Sigma\hat{\pi}$ per horizon — seeds included on both sides, since both are part of the storm's size. E4 Spearman rank correlation of p vs π̂ and average precision against $\hat{\pi} > 0.5$. E5 ten equal-width bins on p: mean p vs mean π̂, and the ECE.

For 914_6, static_plain n50 seed 1: E1 0.0273 (per horizon 0.017 / 0.026 / 0.030 / 0.035); E2 all-sites 0.002–0.007, $\hat{\pi} > 0.05$ sites 0.036–0.053; E3 0.022 / 0.010 / 0.041 / 0.003 (predicted 186.3 / 255.8 / 278.0 / 312.4 vs 190.6 / 258.5 / 290.0 / 313.3); E4 Spearman 0.49–0.79, AP 0.999–1.0; E5 ECE 0.001–0.004. The 96-h calibration: 5,832 sites in the lowest bin with mean p 0.004 vs mean π̂ 0.002; 101 sites in the top bin, 0.963 vs 0.966. Spearman looks low beside AP because it ranks all 6,231 sites, 5,800 of which have $\hat{\pi} \approx 0$ and are ordered by noise; AP asks only whether the ~300 that fail are ranked above the rest.

### 4.7 The pre-registration — the prediction and why I made it

Frozen 27 Oct, commit `3f305d7`, before any code of the direct model existed, with last week's results and the count-floor finding in view and no direct-model output seen.

- **D0 gates.** F1–F5 pass, 24/24 histories, every audit JSON gate-clean. *Why:* no verdict without clean plumbing.
- **D1** ahead of B0 on 13/13 in every cell. *Why:* the minimum bar; any cascade learning clears it.
- **D2** lookup ahead on ≥ 7/12 GISSR storms in every cell; direct ahead on GeoClaw in ≥ 3/4 cells. *Why:* GISSR maps are nested, so the nearest training storm by size is nearly the same map and its π̂ is a near-perfect answer — hard to beat with 50 examples; GeoClaw has a different shape, where the lookup must fail.
- **D3** B2 ahead on ≥ 9/13 in every cell; direct within 1.5× of B2's mean. *Why:* B2 learned from 45,600 examples with conditional structure; a model with 50 examples should lose, but not by more than half again.
- **D4** mean loss falls 17 → 34 → 50 and the second gain is at least half the first; named failure mode: a second gain under 20 % of the first reads as "map diversity, not count, is the limit". *Why:* this is the test of Dr. Lin's "needs more data" and of our "needs diversity" — written so that either outcome has a reading.
- **D5** count error at 96 h within 0.05 of the floor on ≥ 9/13 (static cells). *Why:* Σp is directly supervised by Σπ̂; the direct model should get storm size right.
- **D6** two seeds agree in direction on ≥ 11/13 storms in D2, D3, D4. *Why:* replicability.

---

## 5. What was built — the code map

| File | What it does |
|---|---|
| `config/gnn_direct_split_v1.yaml` | the frozen split as data: train17, new33, train34_added, val7, test13 |
| `src/gnn/data.py` (block behind `GNN_TARGET=marginal`) | `load_depths` (71 storms from two tables), `external_mask` (99 sites), `fragility_probs` (φ via the simulator's function), `storm_context_expected`, `build_direct_x_dict`, `phi_by_horizon`, `marginal_labels` (+halves), `marginal_labels_cached`, `load_direct_split`, `direct_example` |
| `src/gnn/model.py` | `compose_with_fragility`, `soft_bce_with_logits`, `binary_entropy`; the class is untouched |
| `src/gnn/train.py` | `--target marginal`, `--label_mode`, `--train_storms`, `--max_epochs`, `--patience`, `--marginal_cache`; `run_train_marginal`; the per-run path is additions only |
| `scripts/week31_direct_fingerprint_v1.py` | F1–F5, builds the target cache, `--rescore-from` |
| `scripts/week31_direct_audit_v1.py` | E1–E5, B0, B1, B2, floor, gates G1–G6, one JSON per checkpoint |
| `scripts/week31_direct_table_v1.py` | the frozen predictions D0–D6, the tables, the CSV |
| `hpc/fingerprint_gnn_direct_v1.sbatch` | 2 tasks (static, arrival): targets + fingerprint |
| `hpc/campaign_gnn_direct_v1.sbatch` | 24 tasks; refuses to train unless the fingerprint JSON of its label mode says F1–F5 passed and all 70 targets exist |
| `hpc/audit_gnn_direct_v1.sbatch` | 24 tasks; refuses a checkpoint without `history.json` |

Default build unchanged: with `GNN_TARGET` unset, every tensor the two-stage code builds is bit-identical to commit `f45a7b2` (F1), `model.py`'s old text is a prefix of the new file, and `train.py` removes no line.

---

## 6. The checks before training — and the three things they taught us

### 6.1 F1, F4, F5 (no label files needed; ran on the Mac in two minutes)

F1 default build unchanged, in all four switch settings (input widths 9 / 16 / 12 / 22). F4: on four real storms, random weights, static and arrival, timing off and on: $p \geq \varphi_k$ and p non-decreasing on every site; widths 17 and 23; 914_6 156 wet sites and Σφ 91.3, t580w24 216 and 140.3, geoclaw_2050 368 and 295.5, t198w16 737 and 660.1. F5: the loss at $p = \hat{\pi}$ equals the entropy of $\hat{\pi}$ to six decimals (0.460928 on random targets) and is larger at any perturbed p.

### 6.2 F2 — the target recount

An independent count straight from the run records (vectorised, no `data.py` functions) equals `marginal_labels` on 70/70 storms in both label modes: $\hat{\pi}$, the seed frequency and both half-sample estimates, bit for bit.

### 6.3 F3 — φ-consistency, and three lessons

*The check:* over 1,000 runs, a wet non-external site should be a seed with frequency $\approx \varphi_i$. Rule as frozen: within 0.03 on ≥ 99 % of wet sites, `telecom_cluster_03042` excepted (the repository depth table and the frozen campaign disagree there by 0.105 m), external sites at $\varphi = 0$ and never seeding.

*Result:* 26,727 of 26,976 wet non-external site-storms within 0.03 — **99.08 %** (99.10 % with 03042 excepted); externals clean; 03042 beyond 0.03 in five small old storms (173_2, 662_3, 192_7, 708_13, 570_2; gaps 0.10, 0.10, 0.07, 0.03, 0.04 — a real bias, the depth disagreement). The other 244 site-storms beyond 0.03 have gaps ≤ 0.050 in both directions, against an exact binomial expectation of **237.8 (0.88 %)** for 1,000 Bernoulli draws from these φ: sampling noise to the decimal. Identical in both label modes.

**Lesson 1 — the static campaigns never stored the seed set.** The first run crashed: `legacy_v1_n1000` and `gissr48_v1/static` have no `initial_failures` key in any run; the arrival campaigns have it in all 1,000. `data.py` has always fallen back to "the sites stamped at hour 0" for those. Is that the seed set? Under static labels yes if no cascade kill lands at look 0 — and F3 proved it: every storm's seed count and every site beyond 0.03 is identical between the static task (hour-0 fallback) and the arrival task (stored key) — 206.6 seeds for geoclaw_2026 in both, 416.0 for geoclaw_2080. So the static two-stage models have always trained on the true seed mask. Correction on record: the explainer's "stored per run as `initial_failures`" is true for arrival only.

**Lesson 2 — write the pass rule exactly as the pre-registration says.** My first code also demanded that *no* site other than 03042 be beyond 0.03, which the rule never said, and 244 were. F3 came out FAIL on a dataset that met the rule. Fixed by applying the rule as written and printing the binomial expectation next to the observed count; the stored measurements were re-judged with `--rescore-from` (no label re-read); the raw job JSONs are kept beside the rescored ones.

**Lesson 3 — know your noise before you set a gate.** 1,000 draws put 0.9 % of sites beyond 0.03 by chance. A gate with no room for that would always fail.

---

## 7. The 24 training runs — facts

| cell | n | seed 1: epochs / best / val | seed 2: epochs / best / val |
|---|---|---|---|
| static_plain | 17 | 600 (cap) / 562 / 0.0757 | 600 (cap) / 566 / 0.0754 |
| static_plain | 34 | 542 / 491 / 0.0700 | 598 / 547 / 0.0696 |
| static_plain | 50 | 470 / 419 / 0.0694 | 408 / 357 / 0.0694 |
| static_timing | 17 | 561 / 510 / 0.0767 | 561 / 510 / 0.0768 |
| static_timing | 34 | 526 / 475 / 0.0701 | 598 / 547 / 0.0698 |
| static_timing | 50 | 229 / 178 / 0.0713 | 409 / 358 / 0.0696 |
| arrival_plain | 17 | 304 / 253 / 0.1302 | 296 / 245 / 0.1295 |
| arrival_plain | 34 | 343 / 292 / 0.1168 | 343 / 292 / 0.1165 |
| arrival_plain | 50 | 433 / 382 / 0.1143 | 433 / 382 / 0.1145 |
| arrival_timing | 17 | 381 / 330 / 0.1222 | 450 / 399 / 0.1198 |
| arrival_timing | 34 | 392 / 341 / 0.1122 | 302 / 251 / 0.1126 |
| arrival_timing | 50 | 284 / 233 / 0.1083 | 284 / 233 / 0.1076 |

22 stopped early; the two static_plain n17 runs hit the 600 cap with their best at 562 and 566 — still creeping down, so the cap bit by a hair; not re-run, on record. Wall time 5–25 min each. The smoke before production (3 tasks, 5 epochs) showed falling loss and 13 held-out lines per task. Validation loss already separates label modes (static 0.069–0.077, arrival 0.108–0.130) but not training sizes much — a first hint of the flat curve.

---

## 8. The scoring gates — and the float32 lesson (G3)

G1 training finished; G2 the saved validation loss reproduced (to six decimals on all 24); G3 p ≥ φ; G4 p non-decreasing; G5 externals at φ = 0; G6 B2's inputs equal `data.py`'s build.

**G3 failed on all 24 at its first tolerance of $10^{-6}$.** Reproduced on the frozen graph with a model whose cascade logit is pushed very negative (which a trained model does on saturated sites, where there is nothing left to learn): where φ rounds to **exactly 1.0 in float32** — 64 sites in geoclaw_2050, 132 in t198w16, 1,004 site-storms across the 13 test + 7 validation storms — the composition's clamp of φ to $1 - 10^{-6}$ before the log lets p land one float32 step under $1 - 10^{-6}$: **a miss of $7 \times 10^{-8}$** (min $p - \varphi = -1.073 \times 10^{-6}$ on every checkpoint). No site anywhere else misses; none misses by more than $10^{-5}$; in float64 straight from the logits, $1 - p \le 1 - \varphi$ holds on every site (largest excess $-10^{-12}$). The property is intact; the gate left no room for rounding. Tolerance set to $10^{-5}$, the misses recorded by size and by whether φ = 1.0 there (47–199 per checkpoint, nearly all at φ = 1.0), audit re-run, raw JSONs kept. Second re-run: 24/24 `ALL GATES PASS`.

Both corrections (F3 rule, G3 tolerance) touch *gates*, not predictions; the pre-registration says "p ≥ φ" and names no tolerance. Say that before anyone asks.

---

## 9. Results — how to read the table

### 9.1 The columns

Per cell, storms sorted by peak level; INSIDE = GISSR at or below the old training maximum 3.64 m, ABOVE = above it, OTHER = geoclaw_2050. `n17 n34 n50` = the direct model's soft log loss (mean of the two seeds). `B0 phi`, `B1 look`, `B2 2stg`, `floor` as in 4.5. `cnt96` = the n50 model's count error at 96 h; `flr96` = the floor's.

### 9.2 The verdicts, one by one

**D0 PASS.** 24/24 histories; F1–F5 passed in both label modes; every audit JSON gate-clean.

**D1 PASS.** Ahead of B0 on 13/13 in every cell. B0 is 0.38–3.5; the model 0.027–0.18.

**D2 MISS — in the good direction.** Static: lookup ahead on 9/12 GISSR storms, as predicted — but by hairs (914_6: 0.0257 vs 0.0273; t449: 0.0266 vs 0.0283), and because the lookup's answer there is nearly the floor (the nearest nested map is almost the same map). Arrival: lookup ahead on only 6/12 and 5/12 — under arrival labels the timing class of the nearest storm matters and the lookup gets it wrong (270w17: 0.153 vs model 0.122; 808_27: 0.186 vs 0.133; 116w1: 0.155 vs 0.091). GeoClaw: model ahead in 4/4 cells, 0.077 vs 0.31 static. The miss says the direct model is better relative to a table than I expected under arrival labels.

**D3 MISS — in the good direction, and the headline.** B2 ahead on 2 / 4 / 4 / 5 of 13, against the predicted ≥ 9. 13-storm means: direct 0.0690 vs B2 0.0707 (static_plain), 0.0702 vs 0.0706 (static_timing), 0.1156 vs 0.1142 (arrival_plain), 0.1036 vs 0.1054 (arrival_timing). Ratios 0.98 / 0.99 / 1.01 / 0.98. One pass equals a hundred — with 50 training storms against B2's 57. Per horizon in static_plain n50: model 0.055 / 0.056 / 0.086 / 0.079, B2 0.056 / 0.057 / 0.089 / 0.081, floor 0.051 / 0.052 / 0.081 / 0.073. Calibration and ranking are the same too: ECE 0.004–0.005 vs 0.004–0.007, AP 0.997–0.999 vs 0.995–0.998.

**D4 MISS — the named failure mode.** 17 → 34 → 50: 0.0786 → 0.0689 → 0.0690 (static_plain); 0.0808 → 0.0692 → 0.0702; 0.1260 → 0.1142 → 0.1156; 0.1200 → 0.1059 → 0.1036. Second gain over first: −0.01, −0.09, −0.12, +0.16 — all under the 20 % line. And the first gain is not "more maps" either: inside the old level range it is 1–3 % (t449 0.0286 → 0.0281; 914_6 0.0274 → 0.0273), above it 10–35 % (808_27 0.0787 → 0.0711; 605_5 0.1038 → 0.0694; t198 0.1214 → 0.0793). The 17 new storms helped by covering levels the old 17 lacked; once covered, 16 more GISSR maps added nothing. Same pattern as last week's two-stage M1/M3.

**D5 PASS.** Count error at 96 h within 0.05 of the floor on 13/13 in both static cells; the model's errors are 0.001–0.032 against floors of 0.000–0.025. Mean over 13 storms per horizon, static_plain n50: 0.026 / 0.019 / 0.022 / 0.008; static_timing 0.012 / 0.007 / 0.015 / 0.010. The direct model gets storm size right under static labels. Note the floor here (0.000–0.025) is tiny compared with Week 31's per-run count floor (0.50 on t580w24): that one measured run-to-run spread; this one measures

how well the *mean* can be known. The direct model predicts the mean, and the mean of a tipping storm is well defined even when its runs split 350 vs 1,500 — t580w24's count error here is 0.012 vs floor 0.025.

**D6 MISS — where the comparisons are ties.** Seeds agree on direction 13/13 in every arrival comparison and in static D2; they disagree in static D3 (10/13, 6/13) and static D4 (11/13, 8/13) — exactly where model and B2, and n17 and n50 inside the old range, differ by 0.001–0.003, within seed noise. Two seeds cannot agree on the sign of a difference that has none. The miss is informative: it says "tie", not "unstable".

### 9.3 Beyond the verdicts — what else the table says

**Distance to the floor.** Static cells: 13-storm mean 0.069 vs floor 0.064 — within 7 %. Arrival cells: 0.104–0.116 vs 0.084 — 25–40 % above, and the gap is concentrated: geoclaw_2050 0.14–0.18 vs floor 0.078; 605_5 0.116–0.177 vs 0.079; 808_27 0.106–0.133 vs 0.083; t116 0.084–0.091 vs 0.071. The big storms and the differently-shaped storm are where arrival labels are hard.

**GeoClaw under arrival labels gets worse with more GISSR storms** in the plain cell: 0.164/0.158 (n17) → 0.171/0.154 → 0.177/0.182 (n50, both seeds); B2 0.134. Its 96-h count is 20 % low (2,769 vs 3,451). The timing cell holds it at 0.137/0.142 ≈ B2 0.138, count 10 % low. Reading: the model is being pulled toward the GISSR spatial pattern as GISSR storms dominate the training set, and GeoClaw is the one held-out storm shaped differently. This is the spatial-generalisation problem in the only storm that can show it.

**Small-storm counts under arrival labels:** t449 (2.10 m) 96-h count 17–18 % low (294 / 289 vs 353; floor 1.6 %). Static: 2.3 % low. Arrival counts overall: mean per horizon 0.08 / 0.07 / 0.07 / 0.07 (plain), 0.05 / 0.04 / 0.04 / 0.04 (timing) vs floors of 0.005–0.008.

**Timing columns:** under arrival labels they help where it matters — 605_5 0.180/0.175 → 0.115/0.118, 808_27 0.135/0.131 → 0.105/0.107, geoclaw 0.177/0.182 → 0.137/0.142 — and the arrival_timing 13-storm mean is 0.1036 vs 0.1156 plain. Under static labels they change nothing (0.0690 vs 0.0702), as they should: static labels have no timing in them.

**Speed:** 10.4–18.6 ms per storm on an L40S (median ≈ 10.7). The two-stage path needs 100 passes (≈ 1 s); the simulator minutes to half an hour.

---

## 10. What it means

**For Dr. Lin's proposal.** It works, on this data, at parity: one pass reproduces what 100 draws of the two-stage model give, with the same calibration, the same ranking and better storm sizes, from 50 training maps. "Harder to train" did not materialise with the composition supplying the seed part — the network only had to learn the cascade, and 50 storms were enough for that *within the family of maps it saw*.

**For "needs a few hundred flood conditions".** The curve says the number of GISSR maps is not the constraint: 34 was as good as 50, and the 17 → 34 gain was coverage of water levels, not variety. Generating more GISSR storms (the one option the lab has, per Dr. Miura) will add levels we already cover. What would move the model is maps with different spatial patterns — different tracks, different surge shapes — and the GeoClaw-under-arrival result is the warning sign of what happens without them.

**For Dr. Miura, in her terms.** Instead of rolling the dice a hundred times on who the water knocks out and averaging, the model reads the flood map once and says, for each of the 6,231 sites, how likely it is to be down by hour 6, 24, 48 and 96. On the thirteen test storms it is as accurate as the dice version and a hundred times faster; it gets the number of failed sites right to within 3 %. What it still needs is storms shaped differently from the GISSR family — the one Sandy-like map we have (GeoClaw) is where it struggles, and it struggles more as we feed it more GISSR storms.

---

## 11. Limits and open items (say them before they are asked)

1. **13 test storms, 12 of them from the same nested family.** No claim of spatial generalisation; one GeoClaw storm is a hint, not a result.
2. **Arrival labels: 25–40 % above the floor**, worst on the big storms and GeoClaw; small-storm counts 17–18 % low. Static is near-floor; arrival is not.
3. **B2 had 57 training storms, the direct model 50.** The tie is, if anything, conservative in the direct model's favour; but it is a tie, not a win — D6 says so.
4. **Two runs hit the 600-epoch cap** (static_plain n17, both seeds) while still improving slowly.
5. **The direct model gives the mean only.** For a tipping storm like t580w24 (runs split ~350 vs ~1,500 failed sites), the mean is right (count error 0.012) and hides the split; the two-stage model's per-seed-set outputs can show it.
6. **B1 by expected seed count** picks a GeoClaw storm for one GISSR test storm and vice versa; a lookup by level would do better on GISSR and no better on GeoClaw.
7. **Open from before:** 914_6's over-count in the two-stage model (the log-scaled summary), the `predict_cascade.py` wrapper (head and input widths), the depth-table mismatch at 03042 (now measured: 0.03–0.10 in seed frequency on five small storms).
8. **Three corrections on record this week**, none touching a prediction: F3 applied as written; G3 at float32 tolerance; `initial_failures` absent from the static campaigns (and shown harmless).

---

## 12. Questions you will be asked — and the answers

**Dr. Lin: "Isn't p ≥ φ forcing the answer? You are telling the model the seeds."** We are telling it the seed *probability*, which is a deterministic function of the depth map that the simulator itself uses — not a seed draw. The composition is the law of total probability: P(failed) = φ + (1 − φ)·P(failed | not a seed). The network supplies only the second factor. Without the composition the network would have to re-learn the Hazus curves from depth; with it, it learns only what we want to learn, the cascade.

**"Why is the loss not zero for a perfect model?"** Because the target is itself an estimate from 1,000 runs. A site with true probability 0.5 is estimated with standard deviation 0.016. The best possible loss is the entropy of the target plus about 0.001 per uncertain site — that is what the split-half floor measures (0.064 static, 0.084 arrival). The static direct model is within 7 % of it.

**"Last week the two-stage model scored 0.054. Now it scores 0.071. Which is right?"** Both. 0.054 is the per-run loss of a model that sees the realised seeds — it only has to cover the uncertainty left *after* the seeds are known. 0.071 is the same model with the seeds integrated out (100 draws averaged) scored against the storm-level probability, which has more uncertainty in it. The direct model's 0.069 is on the second footing. Compare like with like: inside this table.

**"Why soft BCE and not mean squared error?"** Proper scoring rule (minimised only at the truth), the same family as the two-stage loss, the entropy floor comes for free, and for a fixed prediction it equals the per-run BCE averaged over runs — so the two experiments speak the same language.

**"Fifty examples for a 700,000-parameter network — isn't this memorising?"** Each example is 6,231 × 4 = 24,924 targets, so 50 storms are 1.2 million supervised numbers, and the model is scored on 13 storms it never saw. Early stopping on 7 held-out storms guards the fit. The flat curve says the limit is not overfitting-from-too-few-examples either: 50 did no better than 34.

**"Why did 34 → 50 not help?"** Because the 16 storms added at that step are more of the same: GISSR maps are nested by water level (correlation 0.9998 between level and wet-site count), and the levels were already covered after 34. The 17 → 34 gain is entirely above 3.64 m — levels the old 17 storms lacked. Coverage helped; variety was never added.

**"So what data would help?"** Maps with different spatial patterns: different tracks, different surge shapes. The one differently-shaped storm we have (GeoClaw) is where the model does worst under arrival labels — and it gets worse as GISSR storms dominate training. Candidates outside the lab: more GeoClaw runs with different tracks (the setup exists), NOAA's SLOSH scenario envelopes for

the New York basin (hundreds of track/intensity combinations, behind an emergency-management login), the FEMA Region II synthetic-storm library (159 ADCIRC storms, published grids cover New Jersey). To verify before relying on any.

**“What does the model know about timing under arrival labels?”** In timing cells, the per-site arrival hour, flooded duration and time-to-peak enter the input (divided by 96). They help: arrival_timing 0.104 vs arrival_plain 0.116, and 605_5 goes from 0.18 to 0.12. The seed part φ_ik is gated by arrival ≤ t_k in principle, but every wet site in our storms arrives before the peak, so the gate is always open.

**“Can the two-stage model be dropped now?”** Not yet. It is the only one that gives per-seed-set outcomes (tipping storms); it is slightly better on GeoClaw under arrival labels; and the direct model’s edge is speed and simplicity, not accuracy. The honest statement is parity at 1/100 of the cost on this family of maps.

**“Did you change anything after seeing results?”** The predictions were frozen on 27 Oct before any code existed (`3f305d7`). After the runs I changed two gate tolerances: F3 to the rule as written (my code had added a stricter clause), and G3 from $10^{-6}$ to $10^{-5}$ because of float32 rounding at φ = 1.0. Both are on record with the raw results kept beside the re-judged ones. No prediction, no threshold in D1–D6, no model, no data changed.

**“How do you know your φ is the simulator’s φ?”** F3: over 1,000 runs, the seed frequency matches φ within 0.03 on 99.1 % of the 26,976 wet site-storms; the 0.9 % outside is exactly what binomial noise predicts (237.8 expected, 244 observed); the one real exception is the known 03042 depth disagreement; externals never seed.

**Dr. Miura: “What is the one-sentence takeaway?”** One pass of the model now does what a hundred dice rolls did, equally well and a hundred times faster — and to get better it needs differently-shaped storms, not more of the same.

**“What happens at a site that is never flooded?”** φ = 0, so p = g: pure cascade probability. The 5,832 sites in the lowest calibration bin of 914_6 have mean prediction 0.004 against mean truth 0.002 — the model knows that most of the dry city stays up in a 2-metre storm.

**“Does the curve rise over time for every site?”** Yes, by construction: g is from the hazard read-out (non-decreasing), φ_k is non-decreasing, and their composition is non-decreasing. Checked on every site and horizon of every checkpoint (G4).

---

## 13. Numbers to memorise

- **70 storms** (22 old: 19 GISSR + 3 GeoClaw; 48 new GISSR) × **1,000 runs** × 2 label modes. Split **50 / 7 / 13**. Curve **17 / 34 / 50**. **24 runs**: 4 cells × 3 sizes × 2 seeds. Width **17 / 23**; parameters **699,032 / 701,336**.
- **Composition** $p = 1 - (1 - \varphi)(1 - g)$. **Loss** soft BCE, minimum = entropy; **floor** 0.064 static, 0.084 arrival.
- **Headline:** 13-storm mean 0.069 vs B2 0.071 (static); 0.104 vs 0.105 (arrival_timing). B2 ahead on 2–5 of 13. Speed **≈ 11 ms** per storm vs ≈ 1 s.
- **Curve:** 0.079 → 0.069 → 0.069 (static); 17→34 gain inside the old range 1–3 %, above it 10–35 %; 34→50 gain −1 % to +16 % of the first.
- **Counts:** static 96-h error ≤ 0.032 on 13/13 (floor ≤ 0.025); arrival t449 −17 %, geoclaw −20 % (plain) / −10 % (timing).
- **Lookup:** ahead on 9/12 static GISSR (by ≤ 0.002), 5–6/12 arrival; 0.31–0.41 where it picks the wrong shape.
- **GeoClaw arrival_plain:** 0.16 → 0.17 → 0.18 with more GISSR storms; B2 0.13; floor 0.08.
- **F3:** 99.08 % within 0.03 (99.10 % excepting 03042); 244 noise cases vs 237.8 expected; 03042 gaps 0.03–0.10.
- **G3:** min $p - \varphi = -1.073 \times 10^{-6}$; 1,004 site-storms with φ = 1.0; tolerance $10^{-5}$.
- **Verdicts:** D0 ✓ D1 ✓ D2 ✗(good) D3 ✗(good) D4 ✗(named mode) D5 ✓ D6 ✗(ties). **3 of 7.**

- **Commits:** freeze `3f305d7` → data/train `7b7f373` → scoring `8213375` → F3 seeds `12e6313` → F3 rule `898f966` → G3 (Mac, this morning). Tag on the old branch `week31-two-stage-results` at `f45a7b2`.

---

## 14. The logical flow for tomorrow's deck (one idea per slide)

1. **Last week in one picture** — the two-stage loop: map → φ → draw → network → average.
2. **The question** — Dr. Lin, 15 Oct: can one pass do it?
3. **Why we could test it at once** — the 1,000 runs already give the target. Show $\hat{\pi}$ for one storm as a map.
4. **What goes in, what comes out** — the input row; the composition $p = 1 - (1 - \varphi)(1 - g)$ with the telecom worked example.
5. **How we score it** — soft log loss, the floor, the three baselines, in plain words.
6. **The seven predictions, frozen 27 Oct** — one line each, no numbers.
7. **Result: one pass = one hundred** — the four pairs of means and the speed.
8. **Result: more of the same stops helping** — the 17/34/50 curve, inside vs above the old range.
9. **Where it falls short** — arrival labels vs the floor; GeoClaw getting worse; small-storm counts.
10. **Scorecard** — 3 of 7, with the direction of each miss.
11. **What this says about the flood maps** — diversity, not count; the one GeoClaw storm as the warning; the options.
12. **Next** — what to run, what to ask for.

Backup slides: the F3 result; the G3 rounding story; the `initial_failures` finding; the per-horizon table; calibration for one storm.